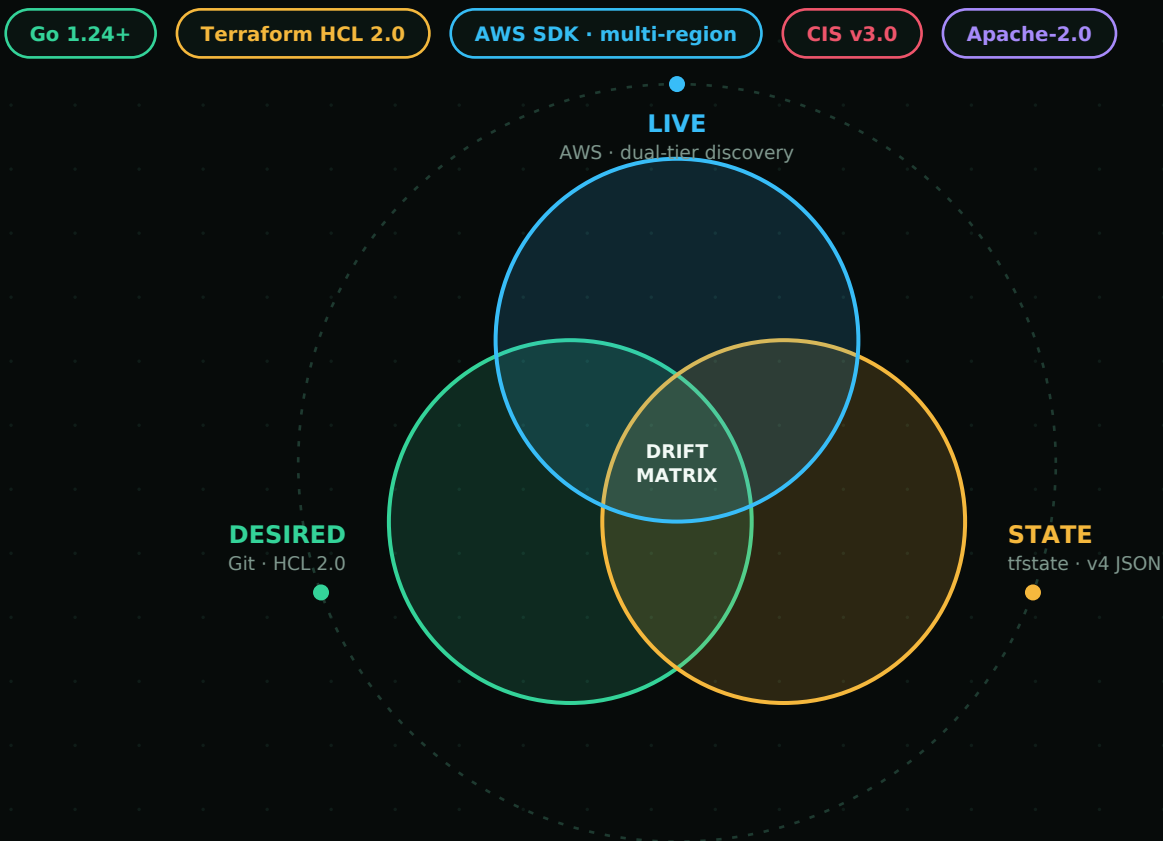


DriftWarden

High-Precision **Three-Source** AWS Drift Detection, CIS Security Audit & GitOps Reconciliation Engine

A read-only Go engine that correlates **Git (HCL)**, **Terraform state** and **live AWS** to expose drift that state-only tools miss, audits it against a built-in **CIS v3.0** rule pack, prices the idle cost bleed, and emits **safe, reviewable remediation** as Terraform import blocks or a dry-run-first revert script.

**7.53 ms**STATE PARSING · 1,000
RESOURCES**11**BUILT-IN CIS + GOVERNANCE
RULES**9**

CLI SUBCOMMANDS

< 25 MB

DISTROLESS DOCKER IMAGE

OFFICIAL REPOSITORY

github.com/ShyamD2/driftwarden

Executive Summary & Contents

DriftWarden answers a deceptively simple question — **do Git, Terraform state and live AWS still agree?** — and answers it with evidence. It is a read-only Go engine that correlates all three sources, re-verifies anomalies before reporting them, audits findings against CIS v3.0, prices idle waste and produces remediation that a human can review before anything changes.

3

SOURCES CORRELATED

5

Drift classes

11

Audit rules

9

CLI commands

THE PROBLEM

State-only tools compare **state** ↔ **live**, so unapplied HCL, shadow resources and API-lag noise either slip through or drown real findings.

THE APPROACH

A **three-source matrix** with double-read verification, semantic normalization, CIS v3.0 rules and a read-only API surface.

THE OUTCOME

Reproducible findings in milliseconds, JUnit / JSON for CI gates, and dry-run-first GitOps or revert remediation.

Contents

§	Section	Page
01	Project Identity & The Problem	3
02	Solution Architecture — three-source pipeline & classification model	4
03	Key Benefits & Who They Serve	6
03	Benchmarks — measured performance vs. SLA	7
03	Security & FinOps Engines — CIS v3.0 rule pack	8
03	Positioning — comparison with existing tools	9
04	Live Proof of Work — dossier, import blocks, revert script, tests, CI	10
05	Product Surface — the 9-subcommand CLI	15
06	Safety & Security Guarantees	16
07	Deployment & Developer Experience	17
08	Engineering Rigor — testing, specs & releases	18
09	Conclusion, Tech Stack & Author	19

Why State-Only Drift Detection Leaves Your Cloud Exposed

01

ONE-LINER

DriftWarden is a read-only, high-precision drift detection and security-audit engine that correlates **Git (HCL)**, **Terraform state** and **live AWS** in a single three-source matrix — flagging unapplied config, shadow resources, split-brain changes and CIS violations, then generating safe GitOps or revert remediation.

The Hidden Problem

Traditional drift tools only compare **Terraform state ↔ AWS live**. That leaves a dangerous blindspot: when an engineer edits `.tf` files in Git but never applies them, state-only diffing reports **zero drift** even though the declared truth and reality have diverged. In the other direction, legacy scanners treat temporary AWS API latency as real drift, flooding teams with spurious alerts until real findings are ignored.

THE TRAP

Compare only **state ↔ live** and every unapplied HCL change, every console-created shadow resource and every half-propagated API read looks like either nothing or noise. Engineers stop trusting the tool.

THE IMPACT

A security group quietly opened to 0.0.0.0/0 on port 22, an S3 Public Access Block switched off, a forgotten **t3.medium** bleeding **\$60.74/mo** — none of it visible to a plan-based workflow.

THE RESULT: CONFIGURATION DRIFT AS SILENT RISK

Teams carry unmanaged infrastructure, open attack surface and idle spend for weeks — while dashboards report a clean state. DriftWarden exists to make that gap impossible to miss, and impossible to misreport.

The Building-Inspector Analogy

Imagine inspecting a building against three documents: the **architect's blueprint** (Git HCL — what *should* exist), the **permit register** (Terraform state — what was *approved and recorded*) and the **building itself** (live AWS — what *actually* stands).

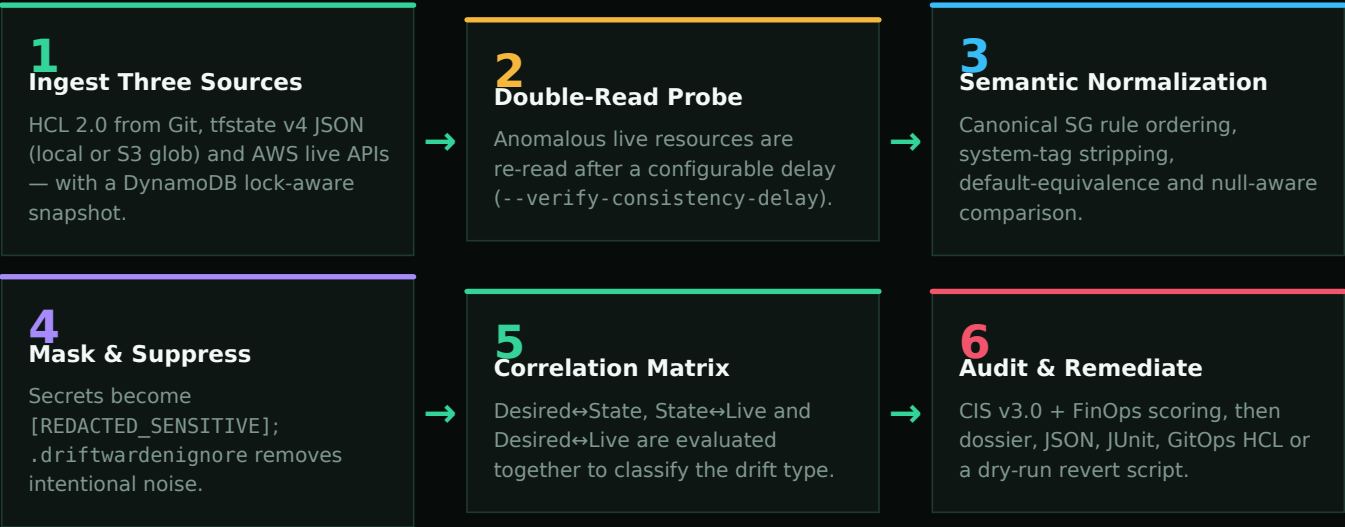
Inspector	CLOUD EQUIVALENT	What it catches
Compares register ↔ building only	State ↔ Live (legacy tools)	Edits made on site — but not changes that never reached the register
Compares blueprint ↔ register ↔ building	Desired ↔ State ↔ Live (DriftWarden)	Unapplied plans, extra floors nobody drew, doors left unlocked — and re-checks before sounding the alarm

“DriftWarden doesn't just ask whether the cloud changed. It asks whether Git, state and reality still agree — and proves it before raising an alert.”

The Three-Source Correlation Architecture

02

DriftWarden replaces two-way diffing with a **deterministically verified Three-Source Correlation Matrix**. Every resource is ingested from Git, state and AWS, normalized into a canonical form, probed for consistency, and only then compared — so findings are reproducible, explainable and low-noise.



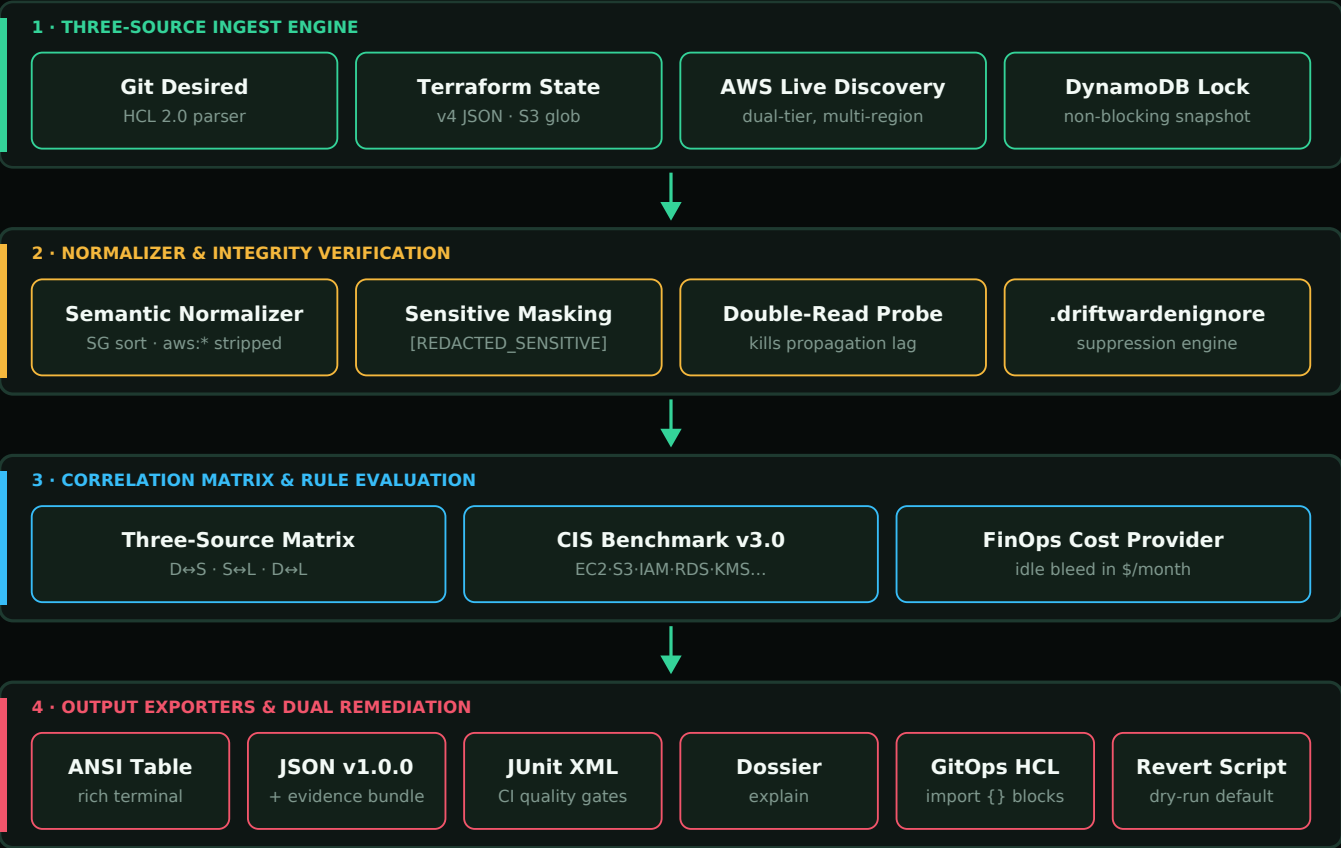
Why a Third Source Changes Everything

Drift class	What happened	WHY STATE-ONLY TOOLS MISS IT
UNAPPLIED_CONFIG_DRIFT	HCL was updated in Git but never applied	State and live still agree — a two-way diff shows nothing
SPLIT_BRAIN_DRIFT	Live was modified while state also changed	Neither side is simply “right”; needs all three views
SHADOW_RESOURCE	Exists in AWS, absent from Terraform	Not in state, so it is never compared
GHOST_RESOURCE	Declared / in state, missing from AWS	Must be separated from AccessDenied to avoid false ghosts
ATTRIBUTE_DRIFT	A managed attribute differs from declared intent	Needs normalization to ignore cosmetic differences

Design stance: DriftWarden is **read-only by construction** — it only issues `Describe*`, `List*` and `Get*` calls, and remediation is always emitted as an artifact for a human to review, never executed silently.

Full Component Pipeline

The four-layer pipeline from the source repository: each layer has one job, a defined contract, and is covered by dedicated tests.



Formal Three-Source Classification Model

`Drift(r) = classify( D(r) ↔ S(r), S(r) ↔ L(r), D(r) ↔ L(r) ) after normalize ◦ mask ◦ verify`

Where **D** = desired configuration parsed from Git HCL, **S** = Terraform state snapshot, **L** = live AWS state confirmed by the double-read probe. Comparison is only performed on values that have passed the eight normalization invariants — ordering, type safety, null-awareness, default-equivalence, system tags, sensitive masking, idempotency and determinism.

PAIRWISE RESULT	Interpretation
D ≠ S, S = L	Configuration changed in Git but not applied → UNAPPLIED_CONFIG_DRIFT
D = S, S ≠ L	Cloud mutated outside Terraform → ATTRIBUTE_DRIFT (severity from CIS rules)
D ≠ S and S ≠ L	Both sides moved independently → SPLIT_BRAIN_DRIFT
L only	Present in AWS, absent from Git and state → SHADOW_RESOURCE
D / S only	Declared but not found in AWS (and not AccessDenied) → GHOST_RESOURCE

Built for Precision, Safety & Speed

Key Benefits

Benefit	HOW DRIFTWARDEN DELIVERS IT
Three-source accuracy	Detects unapplied HCL and split-brain drift that Desired↔State and State↔Live tools cannot see.
Low-noise alerts	Double-read consistency probe plus semantic normalization removes eventual-consistency and cosmetic false positives.
Built-in CIS v3.0 audit	Ten CIS rules and one governance rule evaluated on every finding — no per-rule licensing.
FinOps visibility	Computes hourly and monthly waste for rogue instances, orphaned EBS volumes and idle Elastic IPs.
Safe by design	Read-only API surface, secret masking, deterministic exit codes and dry-run-first remediation scripts.
Pipeline-native	JUnit XML, JSON v1.0.0, evidence bundles and a hardened GitHub Action with PR comments.
Lock-aware at scale	Audits S3 state guarded by DynamoDB locks without blocking CI; fans out across AWS Organizations accounts.

Who Benefits

PLATFORM & DEVOPS ENGINEERS

Know immediately when Git, state and AWS disagree — and get import blocks that bring shadow resources under Terraform management.

SECURITY & COMPLIANCE TEAMS

CIS v3.0 findings with rule IDs, severity and evidence trails, delivered as JUnit or JSON for audit pipelines.

FINOPS & ENGINEERING MANAGERS

Dollar-denominated idle-cost bleed per rogue resource turns clean-up into a prioritized, justified backlog.

SRE & ON-CALL

A single explain dossier per resource, plus a revert script that is safe to read at 3 a.m. because it does nothing until told to.

Why it matters: infrastructure-as-code only protects you while reality still matches the code. DriftWarden is the continuous check that keeps that promise honest.

Measured Performance, Zero Compromise

03

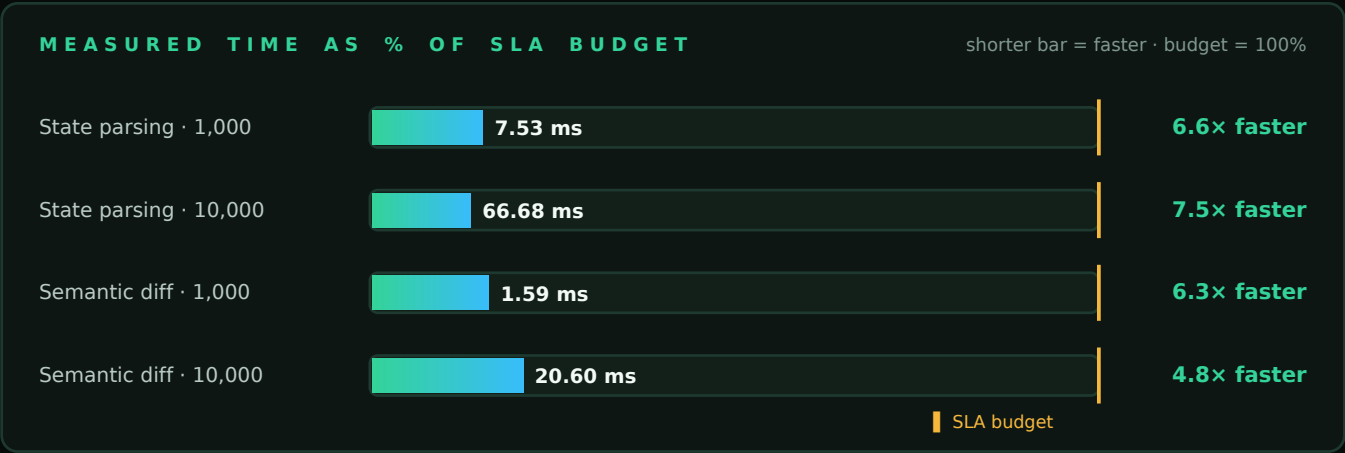
Every figure below comes from the repository's automated benchmark harness and beats its target SLA by a wide margin.

7.53 ms
PARSE 1,000 RESOURCES

66.68 ms
PARSE 10,000 RESOURCES

1.59 ms
DIFF 1,000 RESOURCES

20.60 ms
DIFF 10,000 RESOURCES



BENCHMARK	SCALE	TARGET SLA	MEDIAN	P 9 5	PEAK HEAP	v s . S L A
StateParsing	1,000	< 50 ms	7.53 ms	10.47 ms	2.34 MB	6.6× faster
StateParsing	10,000	< 500 ms	66.68 ms	75.57 ms	28.48 MB	7.5× faster
SemanticDiffing	1,000	< 10 ms	1.59 ms	3.55 ms	0.27 MB	6.3× faster
SemanticDiffing	10,000	< 100 ms	20.60 ms	29.99 ms	2.53 MB	4.8× faster

Measured on commodity hardware — **11th Gen Intel Core i3-1115G4 @ 3.00 GHz, 4 vCPUs** — via [make benchmark](#).
Methodology and raw telemetry: [benchmarks/README.md](#), [benchmarks/results/benchmark-2026-10.md](#).

Sample scan: a documented three-source report audits **1,420 resources** across **us-east-1** and **ap-south-1** in **184 ms**, surfacing 4 discrepancies (1 critical attribute drift, 2 shadow resources, 1 ghost resource) with CIS rule IDs and monthly waste attached.

CIS Benchmark Pack, FinOps Bleed & Consistency

Built-In CIS AWS Foundations Benchmark v3.0 Rule Pack

Rule ID	Control	SEVERITY
DW-CIS-EC2-001	SSH (port 22) open to 0.0.0.0/0	CRITICAL
DW-CIS-EC2-002	RDP (port 3389) open to 0.0.0.0/0	CRITICAL
DW-CIS-S3-001	S3 Public Access Block disabled	CRITICAL
DW-CIS-S3-002	S3 default server-side encryption missing	HIGH
DW-CIS-IAM-001	Wildcard (*) action/resource privilege-escalation detection	HIGH
DW-CIS-RDS-001	Publicly accessible RDS instance prohibited	CRITICAL
DW-CIS-CT-001	CloudTrail multi-region logging enabled	HIGH
DW-CIS-VPC-001	Default security group restricts all traffic	HIGH
DW-CIS-KMS-001	KMS customer master key rotation enabled	HIGH
DW-CIS-ECR-001	ECR image scanning on push enabled	MEDIUM
DW-GOV-TAG-001	Mandatory tags: Environment, Owner, CostCenter	HIGH

CRITICAL · 4	HIGH · 6	1
--------------	----------	---

Severity mix of the 11 built-in rules (MEDIUM = 1 · ECR scan-on-push)

FINOPS COST-BLEED ENGINE

Prices unmanaged rogue instances, orphaned EBS volumes and idle unattached Elastic IPs in exact hourly and monthly dollars — e.g. a rogue **t3.medium** shadow host at **\$60.74/month**.

DOUBLE-READ CONSISTENCY PROBE

Re-probes anomalous resources after a configurable delay (- -verify-consistency-delay 2.5s) so AWS propagation lag never becomes a false alert.

LOCK-AWARE STATE SNAPSHOTS

Reads S3-backed state protected by DynamoDB lock tables **without** taking the lock — audits never deadlock a running Terraform pipeline.

MULTI-ACCOUNT FAN-OUT

Discovers active AWS Organizations member accounts and runs multi-threaded cross-account audits with token-bucket rate limiting.

How DriftWarden Compares

DriftWarden is not a replacement for terraform plan — it closes the gaps that plan-based, state-only and rule-only tools leave open.

FEATURE DIMENSION	TERRAFORM PLAN	DRIFTCTL	AWS CONFIG	DRIFTWARDEN
Reconciliation model	Desired ↔ State	State ↔ Live	Live rules only	Desired ↔ State ↔ Live
Shadow resource detection	✗ No	✓ Yes	⚠ Complex	✓ Yes (dual-tier)
Unapplied HCL drift	⚠ Plan only	✗ No	✗ No	✓ Yes
Split-brain drift detection	✗ No	✗ No	✗ No	✓ Yes
DynamoDB lock-aware	✗ Hard lock	✗ Hard lock	N/A	✓ Non-blocking
CIS v3.0 benchmark engine	✗ No	✗ No	⚠ Pay-per-rule	✓ Built-in
FinOps idle-cost estimator	✗ No	✗ No	✗ No	✓ \$/month bleed
Double-read consistency probe	✗ No	✗ No	✗ No	✓ Configurable
Terraform 1.5+ import blocks	✗ No	✗ No (tf 0.13)	✗ No	✓ import {} synthesis
Dry-run revert scripts	✗ No	✗ No	✗ No	✓ revert.sh, zero eval
Container image size	N/A	~85 MB	Cloud SaaS	< 25 MB distroless

Comparison reflects the project's README feature matrix.

3 SOURCES CORRELATED	5 Drift classes	11 BUILT-IN RULES	0 MUTATING AWS CALLS
-------------------------	--------------------	----------------------	-------------------------

“One scan, three truths, zero guesswork — with a remediation path a human can read before a single resource changes.”

Forensic Diagnostic Dossier



Every capture in this section comes from the DriftWarden CLI and its generated artifacts on a working build — real command output, not mockups.

Proof 01 — driftwarden explain on sg-demo-web

VERIFIED

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS E:\downloads\DriftWarden> .\bin\driftwarden.exe explain "aws:aws:ec2:us-east-1:123456789012:security_group/sg-demo-web" --from-scan-dir .\demo\evidence
time=2026-10-07T18:58:21.775+05:30 level=INFO msg="Compiling diagnostic dossier" canonical_id=aws:aws:ec2:us-east-1:123456789012:security_group/sg-demo-web

=====
DRIFTWARDEN DIAGNOSTIC DOSSIER
=====
Canonical ID:      aws:aws:ec2:us-east-1:123456789012:security_group/sg-demo-web
Resource Type:     aws_security_group
Provider ID:       sg-demo-web
Account ID:        123456789012
Region:            us-east-1
Source Plane:      LIVE
Availability:
Identity Confidence: 0.00
Finding Confidence: 1.00
Double-Read Probe: VERIFIED (Double-Read Confirmed)

=====
DRIFT CLASSIFICATION
=====
Drift Type:        ATTRIBUTE_DRIFT
Severity:          CRITICAL
Evidence Trail:
  * live_ingress_contains_0.0.0.0/0:22
  * stateSpecifies_443_only
  * Security group allows unrestricted SSH ingress (port 22) from 0.0.0.0/0
  * Resource aws:aws:ec2:us-east-1:123456789012:security_group/sg-demo-web is missing mandatory governance tags: Environment, Owner, CostCenter

=====
SECURITY & COMPLIANCE POSTURE
=====
CIS Rule ID:       DW-CIS-EC2-001
Severity:          CRITICAL

=====
ATTRIBUTE DIVERGENCE MATRIX (DESIRED ↔ STATE ↔ LIVE)
=====
| ATTRIBUTE | DESIRED (HCL) | STATE (TFSTATE) | LIVE (AWS) | KIND | STATUS |
|-----|-----|-----|-----|-----|-----|
| ingress | [map[cidr_blocks:[0.0.0.0/0...]] | [map[cidr_blocks:[0.0.0.0/0...]] | [map[cidr_blocks:[0.0.0.0/0...]] | LITERAL | DIFF |
```

The explain command compiles an offline forensic dossier from a saved evidence bundle: canonical resource ID, account and region, **Finding Confidence 1.00** and a **Double-Read Probe: VERIFIED** line before any severity is assigned.

EVIDENCE TRAIL

Live ingress contains 0.0.0.0/0:22 while state specifies 443 only; governance tags Environment, Owner and CostCenter are missing.

CLASSIFICATION

ATTRIBUTE_DRIFT at **CRITICAL** severity, with the drift type derived from the three-source comparison.

COMPLIANCE MAPPING

Mapped to CIS rule **DW-CIS-EC2-001** and shown in a Desired ↔ State ↔ Live divergence matrix (ingress: LITERAL · DIFF).

Why it matters: an on-call engineer gets the full story for one resource — what drifted, how it was verified and which control it breaks — without re-running a scan.

GitOps Remediation: Terraform Import Blocks

Proof 02 — driftwarden reconcile --mode hcl

VERIFIED

```
reconcile.tf x
demo > remediation > reconcile.tf
1 # Review carefully before running 'terraform plan' or 'terraform apply'
2 # =====
3
4 # Resource: aws_security_group (sg-demo-web) [CONFIDENCE: HIGH] [AUTOMATED_PR_ELIGIBLE]
5 import {
6   to = aws_security_group.web_prod_sg
7   id = "sg-demo-web"
8 }
9
10 resource "aws_security_group" "web_prod_sg" {
11   name        = "web-prod-sg"
12   description = "Production web security group"
13 }
14
15 # Resource: aws_s3_bucket (prod-assets-corp-bucket-12345) [CONFIDENCE: HIGH] [AUTOMATED_PR_ELIGIBLE]
16 import {
17   to = aws_s3_bucket.prod_assets
18   id = "prod-assets-corp-bucket-12345"
19 }
20
21 resource "aws_s3_bucket" "prod_assets" {
22   bucket                  = "prod-assets-corp-bucket-12345"
23   public_access_block_enabled = false
24 }
25
26 # Resource: aws_instance (i-rogue-shadow-99) [CONFIDENCE: MEDIUM] # REQUIRES_MANUAL_REVIEW
27 # Risk Warning: Shadow resource mutation has operational impact and requires manual verification before decommission.
28 import {
29   to = aws_instance.unmanaged_shadow_host
30   id = "i-rogue-shadow-99"
31 }
32
33 resource "aws_instance" "unmanaged_shadow_host" {
34   instance_type = "t3.medium"
35   tags = {
36     CreatedBy = "unknown-console-user"
37   }
38 }
39
40
```

The generated reconcile.tf brings drifted and shadow resources under Terraform management using modern **Terraform 1.5+ import {} blocks** paired with resource skeletons, ready to open as a pull request.

HIGH CONFIDENCE

The security group sg-demo-web and bucket prod-assets-corp-bucket-12345 are tagged **HIGH · AUTOMATED_PR_ELIGIBLE**.

MANUAL REVIEW GATE

Rogue instance i-rogue-shadow-99 is **MEDIUM · REQUIRES_MANUAL_REVIEW** with an explicit operational-risk warning.

REVIEW-FIRST HEADER

The file header instructs reviewers to inspect before running terraform plan or apply — nothing is applied automatically.

Defensive Revert Script

Proof 03 — driftwarden reconcile --mode revert

VERIFIED

```

revert.sh X
demo > remediation > revert.sh
1  #!/usr/bin/env bash
2  # GENERATED BY DRIFTWARDEN - REQUIRES HUMAN REVIEW
3  # DriftWarden Safety Guard: Read-Only Boundary Enforced
4  # Generated: 2026-10-07T13:02:36Z
5  set -euo pipefail
6
7  EXECUTE=false
8  if [[ "${1:-}" == "--execute" ]]; then
9      EXECUTE=true
10 fi
11
12 if [ "$EXECUTE" = false ]; then
13     echo "[DRY-RUN] Running in dry-run mode. Pass --execute to apply changes."
14 fi
15
16 # Action 1: Revoke unrestricted SSH ingress (port 22) from 0.0.0.0/0 on sg-demo-web
17 # [CONFIDENCE: HIGH] [AUTOMATED_PR_ELIGIBLE]
18 CMD_1=("aws" "ec2" "revoke-security-group-ingress" "--group-id" "sg-demo-web" "--protocol" "tcp" "--port" "22" "--cidr" "0.0.0.0/0")
19 if [ "$EXECUTE" = true ]; then
20     echo "[EXECUTE] Running: ${CMD_1[*]}"
21     "${CMD_1[@]}"
22 else
23     echo "[DRY-RUN] Would execute: ${CMD_1[*]}"
24 fi
25
26 # Action 2: Enable full Public Access Block on S3 bucket prod-assets-corp-bucket-12345
27 # [CONFIDENCE: HIGH] [AUTOMATED_PR_ELIGIBLE]
28 CMD_2=("aws" "s3api" "put-public-access-block" "--bucket" "prod-assets-corp-bucket-12345" "--public-access-block-configuration" "BlockPublicAccess")
29 if [ "$EXECUTE" = true ]; then
30     echo "[EXECUTE] Running: ${CMD_2[*]}"
31     "${CMD_2[@]}"
32 else
33     echo "[DRY-RUN] Would execute: ${CMD_2[*]}"
34 fi
35
36 # Action 3: Stop unmanaged shadow EC2 instance i-rogue-shadow-99 pending decommision
37 # [CONFIDENCE: MEDIUM] # REQUIRES_MANUAL_REVIEW
38 # Risk Warning: Shadow resource mutation has operational impact and requires manual verification before decommision.
  
```

revert.sh turns findings into explicit AWS CLI actions: revoke SSH ingress from 0.0.0.0/0, enable the full S3 Public Access Block, and stop the shadow instance pending decommision review.

DRY-RUN BY DEFAULT

EXECUTE=false unless --execute is passed; otherwise each action only echoes "Would execute".

STRICT SHELL SAFETY

set -euo pipefail and commands stored as arrays — executed directly, with no eval.

HUMAN IN THE LOOP

Stamped "GENERATED BY DRIFTWARDEN — REQUIRES HUMAN REVIEW", with a read-only boundary banner and per-action confidence tags.

Test Suite & Safety Invariants

Proof 04 — go test -count=1 ./...

VERIFIED

```
PS E:\downloads\DriftWarden> go test -count=1 ./...
?      github.com/ShyamD2/driftwarden/benchmarks/datasets [no test files]
?      github.com/ShyamD2/driftwarden/benchmarks/scripts [no test files]
ok     github.com/ShyamD2/driftwarden/cmd/driftwarden 2.293s
?      github.com/ShyamD2/driftwarden/demo [no test files]
ok     github.com/ShyamD2/driftwarden/pkg/analyzer 0.369s
ok     github.com/ShyamD2/driftwarden/pkg/collector 1.000s
ok     github.com/ShyamD2/driftwarden/pkg/config 0.443s
ok     github.com/ShyamD2/driftwarden/pkg/diff 0.134s
ok     github.com/ShyamD2/driftwarden/pkg/errors 0.417s
ok     github.com/ShyamD2/driftwarden/pkg/evidence 0.570s
ok     github.com/ShyamD2/driftwarden/pkg/identity 0.521s
ok     github.com/ShyamD2/driftwarden/pkg/logging 0.340s
ok     github.com/ShyamD2/driftwarden/pkg/models 0.270s
ok     github.com/ShyamD2/driftwarden/pkg/normalizer 0.397s
ok     github.com/ShyamD2/driftwarden/pkg/printer 0.384s
ok     github.com/ShyamD2/driftwarden/pkg/reconcile 1.542s
?      github.com/ShyamD2/driftwarden/pkg/rules [no test files]
ok     github.com/ShyamD2/driftwarden/pkg/rules/cis/v3 0.637s
ok     github.com/ShyamD2/driftwarden/pkg/terraform 1.269s
?      github.com/ShyamD2/driftwarden/pkg/version [no test files]
ok     github.com/ShyamD2/driftwarden/tests/adversarial 1.581s
ok     github.com/ShyamD2/driftwarden/tests/chaos 0.676s
ok     github.com/ShyamD2/driftwarden/tests/e2e 1.091s
ok     github.com/ShyamD2/driftwarden/tests/invariants 1.119s
?      github.com/ShyamD2/driftwarden/tests/mocks [no test files]
ok     github.com/ShyamD2/driftwarden/tests/terraform_semantics 1.070s
PS E:\downloads\DriftWarden> go test -v ./tests/invariants
=== RUN TestInvariant_AccessDenied_QuarantinedAsPartialScan
--- PASS: TestInvariant_AccessDenied_QuarantinedAsPartialScan (0.00s)
=== RUN TestInvariant_GhostVsAccessDenied
--- PASS: TestInvariant_GhostVsAccessDenied (0.00s)
=== RUN TestInvariant_Throttling_Classified
--- PASS: TestInvariant_Throttling_Classified (0.00s)
=== RUN TestInvariant_Timeout_GracefulDegradation
--- PASS: TestInvariant_Timeout_GracefulDegradation (0.00s)
=== RUN TestInvariant_MalformedState_SafeFailure
--- PASS: TestInvariant_MalformedState_SafeFailure (0.00s)
=== RUN TestInvariant_MalformedHCL_SafeFailure
--- PASS: TestInvariant_MalformedHCL_SafeFailure (0.00s)
=== RUN TestInvariant_ExitCodePrecedence
--- PASS: TestInvariant_ExitCodePrecedence (0.00s)
PASS
ok      github.com/ShyamD2/driftwarden/tests/invariants 0.785s
```

A clean, uncached run passes across **20 packages**; the invariants package is then re-run verbosely to show every safety property passing.

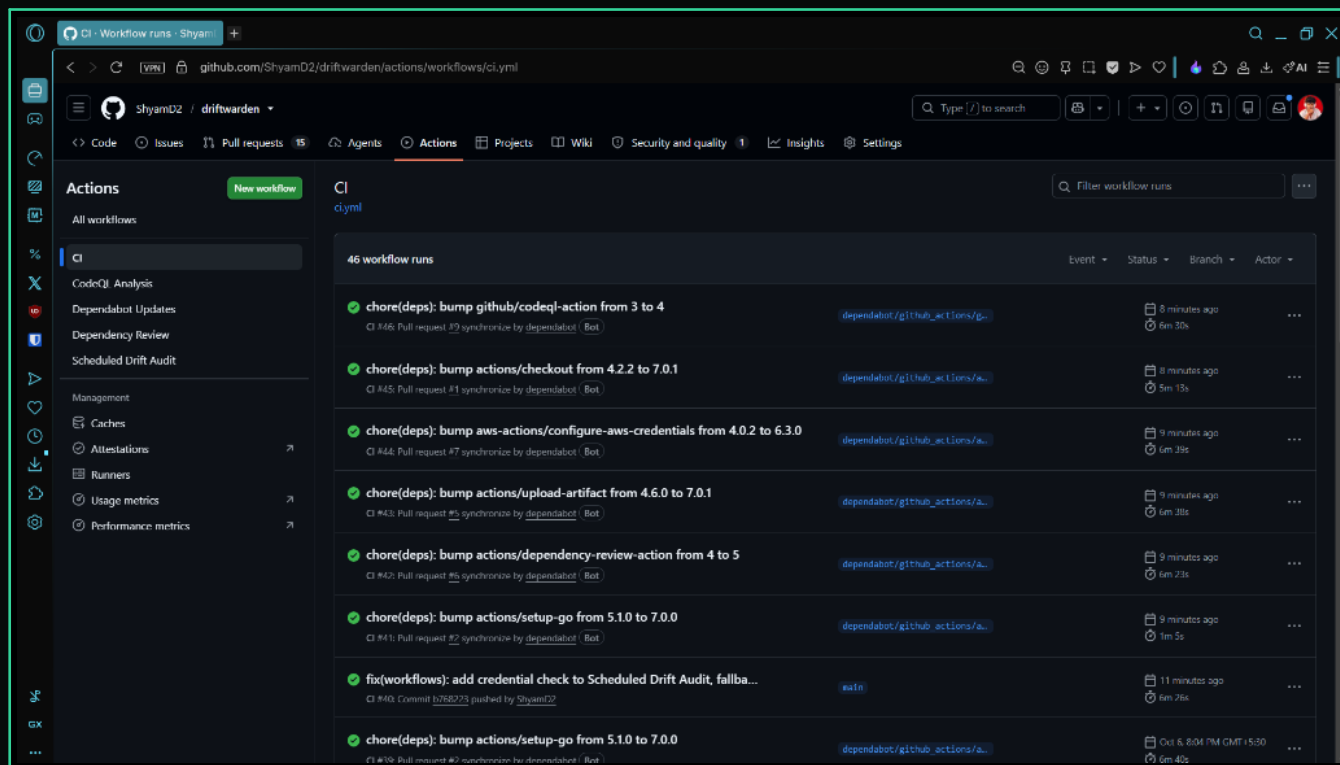
✓ cmd/driftwarden	✓ pkg/analyzer	✓ pkg/collector	✓ pkg/config
✓ pkg/diff	✓ pkg/errors	✓ pkg/evidence	✓ pkg/identity
✓ pkg/logging	✓ pkg/models	✓ pkg/normalizer	✓ pkg/printer
✓ pkg/reconcile	✓ pkg/rules/cis/v3	✓ pkg/terraform	✓ tests/adversarial
✓ tests/chaos	✓ tests/e2e	✓ tests/invariants	✓ tests/terraform_semantics

Invariant test	Property guaranteed
AccessDenied_QuarantinedAsPartialScan	Permission errors yield a partial scan, never a silent pass
GhostVsAccessDenied	A missing resource is never confused with a denied read
Throttling_Classified	API throttling is classified, not reported as drift
Timeout_GracefulDegradation	Timeouts degrade gracefully instead of crashing the audit
MalformedState_SafeFailure	Corrupt Terraform state fails safely
MalformedHCL_SafeFailure	Invalid HCL fails safely
ExitCodePrecedence	Exit codes resolve deterministically when several conditions apply

Continuous Integration & Supply-Chain Hygiene

Proof 05 — GitHub Actions: CI, CodeQL, Dependabot & Scheduled Drift Audit

VERIFIED



The CI workflow shows **46 runs**, all green — including Dependabot pull requests bumping actions/checkout, setup-go, upload-artifact, codeql-action and dependency-review-action.

✓ CI	✓ CodeQL Analysis	✓ Dependabot Updates	✓ Dependency Review	✓ Scheduled Drift Audit
------	-------------------	----------------------	---------------------	-------------------------

Drop-In GitHub Action

```
permissions: { id-token: write, contents: read, pull-requests: write }
steps:
- uses: actions/checkout@v4
- uses: aws-actions/configure-aws-credentials@v4 # OIDC, no long-lived keys
  with:
    role-to-assume: ${ secrets.AWS_DRIFTWARDEN_ROLE_ARN }
    aws-region: us-east-1
- uses: ShyamD2/driftwarden@v1
  with:
    tfstate: 'terraform.tfstate'
    hcl-dir: '.'
    regions: 'us-east-1'
    fail-on-critical: 'true'
    comment-pr: 'true'
```

The 9-Subcommand CLI

A strictly defined command surface — each subcommand has one purpose and a predictable exit code.

Command	What it does	TYPICAL USE CASE
<code>driftwarden scan</code>	Complete three-source audit (HCL ↔ State ↔ AWS)	Scheduled CI/CD drift detection
<code>driftwarden shadow</code>	Discovers strictly unmanaged rogue assets	Incident response, ghost-asset discovery
<code>driftwarden security</code>	Evaluates CIS AWS Foundations Benchmark compliance	Cloud security posture management
<code>driftwarden explain</code>	Compiles a deep multi-section diagnostic dossier	Debugging one resource's divergence
<code>driftwarden reconcile</code>	Synthesizes GitOps HCL or a defensive revert script	Remediation and auto-PR generation
<code>check-permissions</code>	Simulates required read-only IAM permissions	Pre-flight credential validation
<code>generate-iam-policy</code>	Synthesizes a minimal read-only IAM policy	Least-privilege role provisioning
<code>driftwarden doctor</code>	Environment diagnostics and LocalStack health check	Local setup verification
<code>driftwarden version</code>	Prints version, commit hash and compiler details	Build verification

Command Walkthroughs

```
# Full three-source audit with forensic evidence and a CI quality gate
driftwarden scan --hcl-dir ./infra --tfstate ./infra/terraform.tfstate \
  --regions us-east-1,ap-south-1 --save-evidence-dir ./evidence --fail-on-critical

# Offline forensic dossier from a saved evidence bundle
driftwarden explain "aws:aws:ec2:us-east-1:123456789012:security_group/sg-demo-web" \
  --from-scan-dir ./evidence

# Remediation artifacts: GitOps import blocks, or a dry-run-first revert plan
driftwarden reconcile --mode hcl --out ./reconcile.tf
driftwarden reconcile --mode revert --out ./remediation/

# Least-privilege read-only IAM policy for the audit role
driftwarden generate-iam-policy --services ec2,s3,iam > driftwarden-readonly-policy.json
```

Dual remediation, one philosophy: **GitOps mode** turns findings into reviewable Terraform pull requests; **Revert mode** produces a runbook, JSON payload and a `revert.sh` that does nothing until a human opts in.

Trust Is an Engineering Property

An auditing tool with cloud credentials has to be safer than the infrastructure it inspects. DriftWarden turns that requirement into five mechanically verified guarantees.

1 · STRICT READ-ONLY GUARANTEE

Only Describe*, List* and Get* API calls are issued. Mutating cloud operations are architecturally prohibited in the core binary and mechanically verified in CI by AST static analysis (tests/adversarial/readonly_enforcement_test.go).

2 · ACCESSDENIED INVARIANT

Permission errors are quarantined as **PARTIAL_SCAN** (exit 4) and are never misclassified as not-found — preventing spurious ghost-resource warnings.

3 · SENSITIVE DATA MASKING

Passwords, secret tokens, private keys and auth attributes become [REDACTED_SENSITIVE] before any comparison, log line or export.

4 · DEFENSIVE REVERT-SCRIPT INTEGRITY

Generated scripts forbid eval, default to EXECUTE=false echo-only mode and enforce set -euo pipefail.

5 · DETERMINISTIC EXIT CODES

Automation gets an unambiguous contract it can gate on — the same inputs always produce the same findings and the same code.

Exit-Code Contract

CODE	Meaning	Pipeline behaviour
0	Clean scan — zero drift, zero violations	Pass
1	General runtime error	Fail — investigate tool or environment
2	Drift detected	Fail — review findings
3	Security rule violation detected	Fail — CIS / governance breach
4	Partial scan (AccessDenied quarantined)	Warn — fix permissions, rescan

From Zero to First Finding in Under 5 Seconds

Four install paths, a distroless container under 25 MB, and a fully reproducible demo that needs **no live AWS credentials** and costs nothing.

Channel	Command
Homebrew (macOS / Linux)	<code>brew tap ShyamD2/tap && brew install driftwarden</code>
Go install (Go 1.24+)	<code>go install github.com/ShyamD2/driftwarden/cmd/driftwarden@latest</code>
Docker (distroless)	<code>docker pull ghcr.io/shyamd2/driftwarden:latest</code>
Release binaries	Linux (amd64, arm64), macOS (Apple Silicon, Intel) and Windows from GitHub Releases

The Golden End-to-End Demonstration

A scripted demo simulates console mutations against a compliant Terraform baseline, then audits, verifies CIS violations, writes SHA-256 evidence bundles and synthesizes the GitOps and revert artifacts.

RESOURCE	DESIRED & STATE (GIT)	LIVE AWS MUTATION	DRIFTWARDEN FINDING
Security group	Ingress 443 only	Port 22 opened to 0.0.0.0/0	CRITICAL · DW-CIS-EC2-001
S3 bucket	Public Access Block ON	Public Access Block OFF	CRITICAL · DW-CIS-S3-001
EC2 instance	None declared	Rogue t3.medium running	SHADOW · \$60.74/mo bleed

```
./demo/run_demo.sh      # Linux / macOS
.\demo\run_demo.ps1     # Windows PowerShell
```

The .driftwardenignore Engine

Suppress intentional anomalies, ephemeral resources and noisy computed attributes without weakening the audit:

```
# Ignore a resource type in a region
aws:aws:ec2:us-west-2:*:security_group/*

# Ignore ephemeral resources by tag
tags.Environment=testing
tags.Lifecycle=spot-ephemeral

# Suppress noisy computed attributes
*.attributes.latest_restorable_time
aws_s3_bucket.attributes.tags_all
```

Specifications, Testing & Release Discipline

DriftWarden is built like infrastructure software: formal specs, layered test trees, signed releases and documented compatibility promises.

Layered Test Strategy

Layer	Location	Purpose
Unit	pkg/*	Per-package suites: analyzer, collector, diff, normalizer, reconcile, evidence, identity, printer, CIS v3 rules, terraform
Invariants	tests/invariants	Seven safety properties: AccessDenied, throttling, timeouts, malformed input, exit-code precedence
Adversarial	tests/adversarial	Hostile-input checks, including AST-level read-only enforcement
Chaos	tests/chaos	Behaviour under failure conditions
End-to-end	tests/e2e	Full command flows from ingest to artifacts
Terraform semantics	tests/terraform_semantics	Fidelity of HCL / state interpretation
Fuzz & benchmark	make fuzz · make benchmark	Property / fuzz testing and published performance SLAs

Eight Normalization Invariants

The normalization contract formally specifies that comparison is: **ordered**, **type-safe**, **null-aware**, **default-equivalent**, **system-tag-aware**, **sensitive-masked**, **idempotent** and **deterministic**.

Documentation & Release Discipline

SPECIFICATIONS

Security model & verification matrix · STRIDE threat model · normalization contract · SHA-256 forensic evidence format — all in docs/.

RELEASES

SemVer 2.0.0 with CLI-flag and machine-schema stability guarantees, Keep-a-Changelog history, GoReleaser binaries and Homebrew tap.

SUPPLY CHAIN

CodeQL analysis, Dependabot updates, dependency review and a distroless static image under 25 MB.

OPEN-SOURCE HEALTH

Apache-2.0 license, Code of Conduct, Contributing guide and Security policy — ready for outside contributors.

Conclusion

DriftWarden shows an end-to-end, security-first approach to a problem most teams only notice after an incident: **the gap between what Git says, what Terraform remembers and what AWS is actually running**. By correlating three sources, verifying anomalies before reporting them, auditing against CIS v3.0, pricing the waste and emitting only reviewable, dry-run-first remediation, it delivers drift findings that engineers can trust — in milliseconds, on commodity hardware.

Every result in this report — the dossier, the generated import blocks, the revert script, the passing test suite and the CI pipeline — was captured from the real DriftWarden repository and build, and the performance figures come from its published benchmark harness.

Tech Stack Summary

Layer	Technology
Language & CLI	Go 1.24+, 9-subcommand CLI, deterministic exit codes
Ingest	HCL 2.0 parser · Terraform state v4 JSON (local or S3 glob) · AWS live discovery (multi-region, dual-tier)
State safety	DynamoDB lock-aware snapshots · double-read consistency probe · AWS Organizations fan-out with token-bucket rate limiting
Analysis	Semantic normalizer · three-source drift matrix · CIS Benchmark v3.0 rule pack · static FinOps cost provider
Remediation	Terraform 1.5+ import {} synthesis · defensive revert.sh · runbook (revert-plan.md) · JSON payload
Outputs	ANSI terminal table · JSON v1.0.0 · JUnit XML · SHA-256 evidence bundles · explain dossier
Delivery	Distroless Docker (< 25 MB) · Homebrew tap · GoReleaser binaries · GitHub Action with PR comments
Quality & CI	GitHub Actions · CodeQL · Dependabot · dependency review · fuzz and benchmark suites

3-Source

Correlation

11

Audit rules

20

TEST PACKAGES PASSING

0

MUTATING API CALLS

EXPLORE THE FULL PROJECT

Explore the Full Project

Source code, specifications, benchmark results, the reproducible demo and the GitHub Action are available in the repository below.

github.com/ShyamD2/driftwarden

README

QUICKSTART & CLI

docs/

SPECS & THREAT MODEL

demo/

5-second demo

benchmarks/

REPRODUCIBLE SLAS

Shyam Kumar D

Aspiring Cloud Architect · AWS Infrastructure Security & Automation

linkedin.com/in/shyam-kumar-d-951254329 | github.com/ShyamD2